

CROSSPLATFORM MOBILAPPLIKATION

OpenHome

Projektdokumentation

React Native · Expo · TypeScript · Axum REST API

Android og iOS

OpenHome er en mobil fjernbetjening til et privat smart home. Appen styrer lys, TV og højttaler gennem en ekstern Axum API og kan automatisk slukke lyset, når brugeren forlader hjemmet.

Projekt og løsning

OpenHome samler de mest brugte funktioner fra hjemmet i én app. Brugeren konfigurerer serverens adresse og en API-nøgle, hvorefter appen kan sende kommandoer til lys, TV og højttaler. Axum API'en videresender HTTP-requests til to ESP32-enheder: én styrer TV og højttaler med infrarøde signaler, mens den anden styrer en SwitchBot, der tænder og slukker lyset. En valgfri geofence registrerer, når telefonen forlader hjemmet, og sender automatisk en kommando om at slukke lyset.

Klient

React Native og Expo

Backend

Rust og Axum

Kommunikation

REST og JSON

Clean Architecture

Projektet anvender en Clean Architecture-inspireret lagdeling og er opdelt efter ansvar:

- **domain:** datatyper, validering og rene regler.
- **application:** tilstande og koordinering af brugerhandlinger.
- **infrastructure:** REST API, SecureStore, GPS og geofencing.
- **ui:** skærbilleder og genbrugelige React Native-komponenter.

Denne opdeling gør domænelogikken testbar og holder platforms- og netværkskode væk fra brugerfladen. Afhængigheder abstraheres gennem interfaces, så eksempelvis lagring og geofencing kan udskiftes uden at ændre domænereglerne.

Obligatoriske krav

1

Eksterne data og gemt konfiguration

Appen henter IR-status og sender kommandoer til Axum API'en. Base URL, API-nøgle, hjemmeposition og ventende baggrundskommandoer gemmes krypteret med Expo SecureStore og genindlæses mellem sessioner.

2

Selvstændig udvikling

Projektet er udviklet i små, selvstændige dele: opsætning, API-styring, geofence, native Android-modul og swipe-navigation. Den Clean Architecture-inspirerede struktur adskiller domæne, applikation, infrastruktur og UI, så hver del har ét tydeligt ansvar.

3

Valg af udviklingsværktøj

React Native med Expo blev valgt, fordi én TypeScript-kodebase kan bruges på Android og iOS. Expo tilbyder gennemprøvede API'er til placering, sikker lagring og baggrundsopgaver. React Native bruger native UI-komponenter, og Expo kan udvides med native Kotlin-kode, når standardmodulerne ikke er nok.

4

Ekstern datakilde

Appen kommunikerer med en ekstern REST API via `fetch`. Den læser dynamisk tilgængelige IR-kommandoer og sender POST-kald til TV, højttaler og lys. Axum API'en sender derefter HTTP-requests til to ESP32-enheder: en infrarød fjernbetjening til TV og højttaler samt en SwitchBot-controller til lyset. JSON-svar behandles som ukendte data og valideres før brug.

5

Samspil med enheden

Appen anmoder om præcis GPS-position og tilladelse til baggrundsplacering. Positionen bruges til at oprette en Home Geofence. Android har desuden et lokalt Kotlin-modul baseret på `LocationManager`; iOS anvender Expo Location.

6

Asynkrone processer

Netværk, sikker lagring, GPS og baggrundsopgaver anvender Promises samt `async/await`. Det forhindrer langsomme operationer i at blokere brugerfladen. Samtidige kommandoer følges uafhængigt, så eksempelvis TV og højttaler kan styres uden at låse hele appen.

7**Optimering og ressourceforbrug**

Geofencing udføres af operativsystemet i stedet for kontinuerlig GPS-polling. API-status hentes ved opstart og ved et aktivt genforsøg, ikke løbende. Dublerede knaptryk afvises, requests afbrydes efter fem sekunder, og skærmbilleder deler samme indlæste IR-status.

8**Sikkerhed**

Alle API-kald kræver en Bearer API-nøgle. Nøglen vises skjult og gemmes i platformens SecureStore. URL, nøgle, API-svar, koordinater og radius valideres. HTTPS bør bruges uden for et betroet lokalnet; den nuværende udviklingsopsætning tillader også HTTP til en lokal server og skal derfor ikke bruges over et ubeskyttet netværk.

9**Dokumentation og test**

Projektet har 24 automatiske tests fordelt på seks testfiler. De dækker validering, state transitions, API-svar, geofence, fejlhåndtering og retry. TypeScript typecheck, Expo Doctor samt Android- og iOS-bundling er bestået. Screenshots er vist i dette dokument. Demo-video, manuel testrapport og brugerfeedback vedlægges afleveringen.

10**Mundtlig præsentation**

Præsentationen planlægges til 10-15 minutter: problem og målgruppe, live demo, arkitektur, API og sikkerhed, geofencing, test, udfordringer og refleksion. Demoen viser opsætning, fjernbetjening og Leave home-automatik.

11**Problemløsning og refleksion**

Expo-geofencing på Android afhænger normalt af Google Play Services. For at understøtte enheder uden Google Play blev der udviklet et Kotlin-modul med Androids `LocationManager`. Modulet genbruger samme TypeScript-applikationslogik. Løsningen øger kompleksiteten, men undgår løbende GPS-opdateringer og bevarer lavt batteriforbrug.

12**Metodevalg**

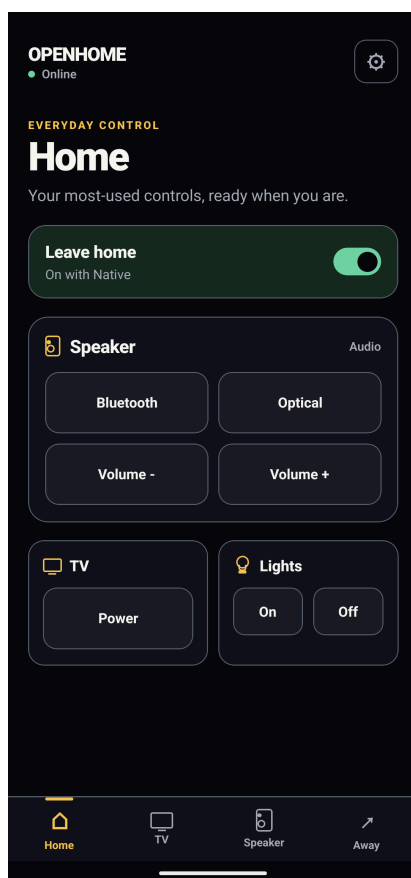
Arbejdet fulgte en enkel, Kanban-inspireret og iterativ metode. Funktioner blev opdelt i små vertikale leverancer, implementeret én ad gangen og kontrolleret med tests og builds. Metoden passede til et enkeltmandsprojekt, fordi prioriteringer kunne ændres uden faste sprintceremonier.

Valgfri funktionalitet

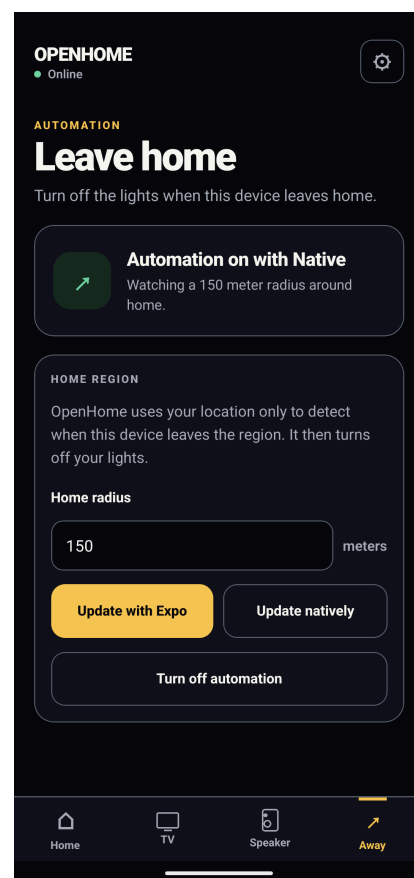
Følgende valgfri funktioner er implementeret:

- **Dynamiske lister og brugerinput:** API'en bestemmer, hvilke fjernbetjeningskommandoer der er tilgængelige. Brugeren indtaster server, API-nøgle og radius.
- **Lokal lagring:** SecureStore gemmer konfiguration og geofence krypteret.
- **Touch og interaktivitet:** knapper, switch, faner og vandret swipe-navigation.
- **Sensorer:** GPS og geofencing bruges til Leave home-automatik.
- **Udvidet API-integration:** dynamiske JSON-data og Bearer-authentication.

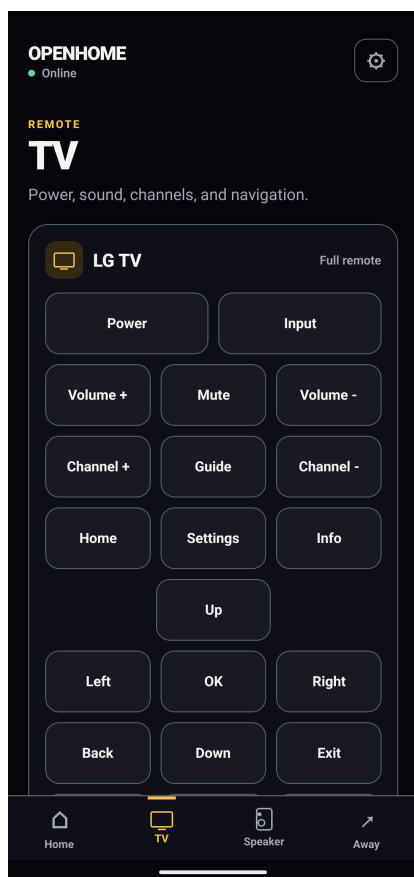
Skærbilleder



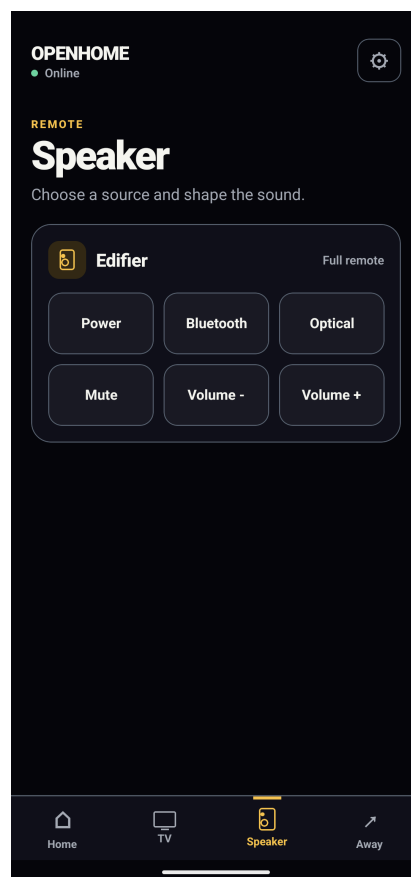
Figur 1: Home med hurtig adgang



Figur 2: GPS-baseret automatik



Figur 3: Dynamisk TV-fjernbetjening



Figur 4: Højttalerstyring

Kort testrapport

Test	Formål	Resultat
TypeScript	Kontrol af typer og interfaces	Verificeret
Vitest	24 tests i seks testfiler	Verificeret
Expo Doctor	20 projekt- og afhængighedstjek	Verificeret
Android export	JavaScript-bundle til Android	Verificeret
iOS export	JavaScript-bundle til iOS	Verificeret
Fysisk enhed	Tilladelser, GPS, baggrund og API	Verificeret

Forslag til manuel brugertest

1. Gem en gyldig serveradresse og API-nøgle, genstart appen, og kontrollér at opsætningen huskes.
2. Send kommandoer til TV, højttaler og lys, og kontrollér den fysiske reaktion.
3. Afvis og godkend placeringstilladelser, og kontrollér fejlbeskederne.
4. Opret en Home Geofence, forlad området, og kontrollér at lyset slukkes.
5. Afbryd netværket, udløs en kommando, og kontrollér fejl samt retry efter genåbning.

Præsentationsplan

Indhold	Tid
Idé, målgruppe og problem	1 min.
Live demo af de vigtigste funktioner	4 min.
Arkitektur, API og asynkron kode	3 min.
GPS, geofencing, optimering og sikkerhed	3 min.
Test, udfordringer og refleksion	2 min.
Spørgsmål	2 min.

Afleveringstjek

- **Klar** Kildekode og screenshots.
- **Klar** Automatisk test, typecheck og platform-bundles.
- **Klar** Projektdokumentation og præsentationsplan.